

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_a6bedd49-683\agent-a79a43035411d3505.jsonl`
- SHA-256: `f6c326bad8d85a412604fb598e868d63f6248626f070d60c655e2e50ec5a4263`
- Source modified: `2026-07-06T23:58:03+00:00`
- Imported at: `2026-07-08T16:00:07+00:00`
- project: `wf\_a6bedd49-683`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T23:56:40.273Z)

Reviewing GitHub PR #55 "F6: persist policy decision traces" (branch feat/f6-policy-decisions -> main). ADR 0012 F6.
+387/-11, 7 files. Checked-out copy at C:/Users/avidu/Projects/_wt-pr55 (read there or 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f6-policy-decisions:<path>').
Run repros: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr55/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>.

WHAT F6 DOES:

- db.py: new policy_decisions table in BOTH _SCHEMA and @_migration(5)
_migration_005_policy_decisions (both CREATE TABLE IF NOT EXISTS).
Schema: (id PK, content_item_id FK content_items, policy_id FK policies, passed INT, trace_json TEXT, evaluated_at TEXT, UNIQUE(content_item_id, policy_id)).
Repos: record_policy_decision (UPSERT on (content_item_id,policy_id):
passed/trace_json/evaluated_at), get_policy_decision, list_policy_decisions_for_content.
_policy_trace_json = json.dumps(trace, sort_keys, compact). _row_to_policy_decision json.loads + validates list.
- models.py: PolicyDecision (frozen). policy.py: stage_results_to_trace(results) -> [{stage_name,passed,reason,metadata}].
- telethon_client.py: _evaluate_policy_stages now returns (passed, results, policy_id).
_record_policy_decision(inserted, policy_id, policy_passed, policy_results):
guards 'if not policy_results or policy_id is None or inserted.id is None: return'; else
db.record_policy_decision(content_item_id=inserted.id, ...).
Called in on_new_message AND backfill AFTER insert_video (so the FK content_item exists).
on_new_message/backfill are '# pragma: no cover'.

VERIFIED by main reviewer: gate green (551 passed @ 94.34%); migration idempotent + reaches user_version=5 on a legacy v1 DB; a full pipeline trace serializes + persists; UPSERT is idempotent (1 row on re-record); policy decisions are stored SEPARATELY from delivery_attempts (addresses an earlier Track C finding).
Note deviations/limits already spotted: schema is one-row-per-decision-with-JSON-trace (ADR literally proposed per-stage rows); NO TTL/cleanup (ADR open question); metadata-rejected (Tier-1 FilterEngine) items get NO policy_decision row (only the evaluate_pipeline Tier 2-4 trace is recorded).

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR's diff. No praise. Concrete failure scenario, ranked.
Severity: high (migration data loss/corruption or a write bug that crashes ingestion/persists wrong data), medium (real latent bug/FK/serialization crash), low (note/deviation/growth).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read cited code, run a repro with
PYTHONPATH=C:/Users/avidu/Projects/_wt-pr55/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe.
CONFIRMED only if the defect genuinely ships in THIS PR's diff and produces the described wrong

behaviour. REFUTED if handled / can't occur / pre-existing / acceptable-deferred. PLAUSIBLE if unsettled. Corrected severity (migration data loss/corruption = high; latent serialization crash = medium; note = low).

FINDING (persist_wiring): Non-serializable stage metadata crashes ingestion via uncaught json.dumps TypeError on the policy-decision write path

severity medium | src/tg_video_filter/db.py:1391

summary: record_policy_decision -> _policy_trace_json does json.dumps(trace) with no error handling; any StageResult whose metadata contains a non-JSON-serializable value raises TypeError that propagates uncaught out of _record_policy_decision and out of both call sites (on_new_message and the backfill message loop have no try/except around it), crashing per-message ingestion / aborting the whole backfill.

scenario: A future ContentFetcher (F5) or budget path returns/raises metadata carrying a non-JSON value (e.g. a datetime, set, bytes, or arbitrary object) -- the most concrete exposed path is DeepInspectionStage at policy.py:763 which splats `\*\*exc.metadata` from a caller-supplied PolicyBudgetExceeded straight into the StageResult. stage_results_to_trace copies result.metadata verbatim into the trace, then record_policy_decision calls json.dumps and raises `TypeError: Object of type X is not JSON serializable`. In on_new_message (telethon_client.py:342) there is no surrounding try/except, so the handler dies; in backfill the try at backfill.py:147 only catches FloodWaitError, so the TypeError bubbles to the outer `except Exception` (backfill.py:464) and aborts the entire user backfill. Reproduced: stage_results_to_trace([StageResult.pass_('s','ok',obj=object())]) then db.record_policy_decision(...) raises TypeError uncaught. NOTE: not triggerable by current F1-F5 stages -- every present metadata builder emits only str/int/bool/list-of-str (topic.py list(...), url_reputation_url_reputation_metadata, caption_caption_safety_metadata), so this is latent/defensive, same class as the F5 fetcher-error lesson.

evidence: db.py:1387-1390 `\_policy\_trace\_json` = json.dumps(trace, separators, sort_keys) with no try/except; policy.py:838-845 stage_results_to_trace copies result.metadata unchanged; policy.py:757-764 DeepInspectionStage splats untrusted `\*\*exc.metadata`; telethon_client.py:342-348 and 464-475 call the helper with no try/except; backfill.py:147 try only pairs with `except FloodWaitError` at :327. Repro output: `record\_policy\_decision raised TypeError: Object of type object is not JSON serializable`.

rec: Make the trace write defensive so an audit-log serialization failure can never crash the ingestion/backfill main path: either wrap the json.dumps in _policy_trace_json with a fallback (json.dumps(..., default=str)) or wrap the db.record_policy_decision call inside _record_policy_decision in try/except that logs and swallows (best-effort audit, matching the F5 fetcher-error handling), so a bad-metadata stage degrades to a missing/repr'd trace row rather than a dropped message or aborted backfill.

2. assistant (2026-07-06T23:56:42.285Z)

I'll verify this finding by reading the cited code and running a repro.

3. user (2026-07-06T23:56:43.771Z)

```

1380         config_json=_policy_config_dict_to_json(config),
1381         is_default=True,
1382         created_at=(existing.created_at if existing is not None else _now()),
1383         updated_at=_now(),
1384     ),
1385 )
1386
1387
1388 # --- PolicyDecision repos (ADR 0012, Track F) -----
1389
1390
1391 def _policy_trace_json(trace: list[dict[str, object]]) -> str:
1392     """Encode a policy stage trace deterministically."""
1393     return json.dumps(trace, separators=(",", ":"), sort_keys=True)
1394
1395
1396 async def record_policy_decision(
1397     conn: aiosqlite.Connection,
1398     *,
1399     content_item_id: int,
1400     policy_id: int,

```

```

1401         passed: bool,
1402         trace: list[dict[str, object]],
1403         evaluated_at: datetime | None = None,
1404     ) -> PolicyDecision:
1405         """Persist or replace the policy evaluation trace for a content item."""
1406         evaluated_at = evaluated_at or _now()
1407         trace_json = _policy_trace_json(trace)
1408         await conn.execute(
1409             """INSERT INTO policy_decisions
1410                 (content_item_id, policy_id, passed, trace_json, evaluated_at)
1411                 VALUES (?, ?, ?, ?, ?)
1412                 ON CONFLICT(content_item_id, policy_id) DO UPDATE SET
1413                     passed=excluded.passed,
1414                     trace_json=excluded.trace_json,
1415                     evaluated_at=excluded.evaluated_at""",
1416             (
1417                 content_item_id,
1418                 policy_id,
1419                 int(passed),
1420                 trace_json,
1421                 evaluated_at.isoformat(),
1422             ),
1423         )
1424         await conn.commit()
1425         decision = await get_policy_decision(conn, content_item_id, policy_id)
1426         assert decision is not None
1427         return decision
1428
1429
1430     async def get_policy_decision(
1431         conn: aiosqlite.Connection,
1432         content_item_id: int,
1433         policy_id: int,
1434     ) -> PolicyDecision | None:
1435         """Read a policy decision by its natural key."""
1436         conn.row_factory = aiosqlite.Row
1437         async with conn.execute(
1438             """SELECT * FROM policy_decisions
1439                 WHERE content_item_id = ? AND policy_id = ?""",

```

4. user (2026-07-06T23:56:44.470Z)

```

750         "deep inspection fetcher missing",
751         missing_fetcher=True,
752     )
753
754     assert ctx.budget is not None
755     try:
756         return await ctx.budget.inspect(ctx.fetcher, ctx.content_item)
757     except PolicyBudgetExceeded as exc:
758         return StageResult.fail(
759             self.stage_name,
760             "deep inspection budget exceeded",
761             budget_exceeded=True,
762             exceeded_budget=exc.exceeded_budget,
763             **exc.metadata,
764         )
765     except Exception as exc:
766         log.warning(
767             "deep inspection fetcher failed policy_id=%s",
768             ctx.policy.id,
769             exc_info=True,
770         )
771         return StageResult.fail(
772             self.stage_name,

```

```

773         "deep inspection fetcher failed",
774         fetcher_error_type=type(exc).__name__,
775         fetcher_error=str(exc),
776     )
777
778
779 def _deep_inspection_budget_result(ctx: PolicyContext) -> StageResult | None:
780     budget = ctx.budget
781     if budget is None:
782         return StageResult.fail(
783             DeepInspectionStage.stage_name,
784             "deep inspection budget missing",
785             missing_budget=True,
786         )
787     if budget.max_bytes_per_item <= 0 or budget.timeout_s <= 0 or budget.max_concurrent
788     <= 0:
789         return StageResult.fail(
790             DeepInspectionStage.stage_name,
791             "deep inspection budget invalid",
792             invalid_budget=True,
793             max_bytes_per_item=budget.max_bytes_per_item,
794             timeout_s=budget.timeout_s,
795             max_concurrent=budget.max_concurrent,
796         )
797     size_bytes = ctx.content_item.size_bytes
798     if size_bytes is None:
799         return StageResult.fail(
800             DeepInspectionStage.stage_name,
801             "deep inspection budget cannot be enforced without content size",
802             missing_size_bytes=True,
803             max_bytes_per_item=budget.max_bytes_per_item,
804         )
805     if size_bytes <= 0:
806         return StageResult.fail(
807             DeepInspectionStage.stage_name,
808             "deep inspection budget cannot be enforced with non-positive content size",
809             invalid_size_bytes=size_bytes,
810             max_bytes_per_item=budget.max_bytes_per_item,
811         )
812     if size_bytes > budget.max_bytes_per_item:
813         return StageResult.fail(
814             DeepInspectionStage.stage_name,
815             "deep inspection budget exceeded",
816             budget_exceeded=True,
817             exceeded_budget="max_bytes_per_item",
818             size_bytes=size_bytes,
819             max_bytes_per_item=budget.max_bytes_per_item,
820         )
821     return None
822
823 async def evaluate_pipeline(
824     ctx: PolicyContext,
825     stages: list[PolicyStage],
826 ) -> tuple[bool, list[StageResult]]:
827     """Evaluate stages in order and short-circuit on the first failure."""
828     results: list[StageResult] = []
829     for stage in stages:
830         result = await stage.evaluate(ctx)
831         results.append(result)
832         if not result.passed:
833             return False, results
834     return True, results
835
836

```

```

837     def stage_results_to_trace(results: list[StageResult]) -> list[dict[str, object]]:
838         """Convert in-memory stage results to a JSON-serializable trace."""
839         return [
840             {
841                 "stage_name": result.stage_name,
842                 "passed": result.passed,
843                 "reason": result.reason,
844                 "metadata": result.metadata,
845             }
846             for result in results
847         ]
848
849

```

5. assistant (2026-07-06T23:56:46.883Z)

Now let me look at the call sites in telethon_client.py and backfill.py, plus StageResult definition.

6. user (2026-07-06T23:56:47.925Z)

```

320         except sqlite3.IntegrityError:
321             # Lost a dedup race: a concurrent delivery of the same file
322             # (same document_id or same message instance) won the insert.
323             # The three-tier check above already classified this as new, so
324             # the collision means another task is processing (or has
325             # processed) it ? log and skip.
326             log.info(
327                 "video_dedup_race user=%s channel=%s msg=%s doc_id=%s",
328                 user.id,
329                 channel_id,
330                 message_id,
331                 document_id,
332             )
333             return
334         log.info(
335             "video_seen user=%s channel=%s msg=%s matched=%s reasons=%s",
336             user.id,
337             channel_id,
338             message_id,
339             matched,
340             reasons,
341         )
342         await _record_policy_decision(
343             conn,
344             inserted=inserted,
345             policy_id=policy_id,
346             policy_passed=policy_passed,
347             policy_results=policy_results,
348         )
349
350         if matched:
351             # Build link text for LINK/BOTH modes (None for FORWARD).
352             link_text = None
353             if delivery_mode != DeliveryMode.FORWARD:
354                 channel_title = _channel_title(event)
355                 link_text = build_link_text(
356                     message_link=getattr(message, "link", None),
357                     duration_s=meta.duration_s,
358                     height=meta.height,
359                     size_bytes=meta.size_bytes,
360                     channel_title=channel_title,
361                 )
362             await deliver(
363                 client,

```

```

364         message,
365         conn,
366         inserted.id,
367         delivery_mode,
368         link_text=link_text,
369         target=delivery_target,
370     )
371
372     return on_new_message
373
374
375 # --- helpers -----
376
377
378 def _extract_document_id(message: Message) -> int | None:
379     """Extract Telegram's ``document.id`` from a video message.
380
381     Returns None if the message has no document media or the id is absent.
382     This is the most reliable content-based dedup key: Telegram reuses the
383     same ``document.id`` when a file is forwarded across channels.
384     """
385     media = getattr(message, "media", None)
386     if isinstance(media, MessageMediaDocument):
387         media = media.document
388     if isinstance(media, Document):
389         return getattr(media, "id", None)
390     return None
391
392
393 def _message_text(message: object) -> str | None:
394     """Return caption/message text normalized for storage, or None if empty."""
395     text = getattr(message, "message", None)
396     if text is None:
397         text = getattr(message, "raw_text", None)
398     if not isinstance(text, str):
399         return None
400     text = text.strip()
401     return text or None
402
403
404 def _content_item_preview(
405     account_id: int,
406     source: Source,
407     message_id: int,
408     document_id: int | None,
409     meta: VideoMeta,
410     text: str | None,
411     matched: bool,
412 ) -> ContentItem:
413     """Build the ContentItem shape used by policy stages before insert."""
414     assert source.id is not None
415     return ContentItem(
416         account_id=account_id,
417         source_id=source.id,
418         provider=source.provider,
419         provider_message_id=str(message_id),
420         content_type="video",
421         text=text,
422         document_id=str(document_id) if document_id is not None else None,
423         duration_s=meta.duration_s,
424         width=meta.width,
425         height=meta.height,
426         size_bytes=meta.size_bytes,
427         mime_type=meta.mime_type,
428         dedup_key=db.compute_dedup_key(

```

```

429         provider=source.provider,
430         document_id=str(document_id) if document_id is not None else None,
431         source_id=source.id,
432         provider_message_id=str(message_id),
433         size_bytes=meta.size_bytes,
434         duration_s=meta.duration_s,
435         height=meta.height,
436         width=meta.width,
437     ),
438     matched=matched,
439 )
440
441
442 async def _evaluate_policy_stages(
443     conn: aiomysqlite.Connection,
444     account_id: int,
445     source: Source,
446     content_item: ContentItem,
447 ) -> tuple[bool, list[StageResult], int | None]:
448     """Evaluate ADR 0012 policy stages for a candidate item."""
449     policy = await db.get_default_policy(conn, account_id)
450     if policy is None:
451         return True, [], None
452     # DeepInspectionStage is dormant unless ``deep_inspect`` is enabled. Any
453     # user-facing enable command must land with fetcher/budget wiring.
454     passed, results = await evaluate_pipeline(
455         PolicyContext(content_item=content_item, source=source, policy=policy),
456         [
457             TopicPreferenceStage(),
458             UrlReputationStage(),
459             CaptionSafetyStage(),
460             DeepInspectionStage(),
461         ],
462     )
463     return passed, results, policy.id
464
465
466 async def _record_policy_decision(
467     conn: aiomysqlite.Connection,
468     *,
469     inserted: VideoRecord,
470     policy_id: int | None,
471     policy_passed: bool,
472     policy_results: list[StageResult],
473 ) -> None:
474     """Persist a policy trace after the content item has been inserted."""
475     if not policy_results or policy_id is None or inserted.id is None:
476         return
477     await db.record_policy_decision(
478         conn,
479         content_item_id=inserted.id,

```

7. user (2026-07-06T23:56:52.307Z)

```

479         content_item_id=inserted.id,
480         policy_id=policy_id,
481         passed=policy_passed,
482         trace=stage_results_to_trace(policy_results),
483     )
484
485
486 def _peer_id(peer: object) -> int:
487     """Best-effort numeric id for the peer of a message.
488
489     Telegram represents channel/group/user peers as different TL types; we

```

```
490         collapse them to a single int for dedup/storage. The exact subclass isn't
491         important ? only that the id is stable per chat.
492         """
493         # Each Peer* type has a field like channel_id / chat_id / user_id.
```

8. user (2026-07-06T23:56:52.926Z)

```
140         await progress_callback(f"? Scanning #{title} (last {days} days)...")
141
142         dialog_videos = 0
143         dialog_total = 0
144         dialog_matched = 0
145         last_progress = asyncio.get_event_loop().time()
146
147         try:
148             # NOTE: we do NOT use offset_date because it gets messages OLDER
149             # than the given date. Instead we iterate newest-first and manually
150             # break when we cross the cutoff boundary.
151             async for message in client.iter_messages(
152                 dialog.entity,
153                 wait_time=_MIN_WAIT_BETWEEN_BATCHES,
154                 reverse=False, # newest-first
155             ):
156                 # Stop when we reach messages older than the cutoff.
157                 msg_date = getattr(message, "date", None)
158                 if msg_date is not None and msg_date < cutoff:
159                     break
```

9. assistant (2026-07-06T23:56:55.339Z)

Now let me check the StageResult definition and metadata handling, and check where backfill records the decision.

10. user (2026-07-06T23:56:56.977Z)

```
41:     _record_policy_decision,
235:         except Exception:
291:         await _record_policy_decision(
327:         except FloodWaitError as e:
366:         except Exception:
464:     except Exception:
```

11. user (2026-07-06T23:56:57.878Z)

```
26:     - ``LINK``: send a text message with a t.me link + metadata summary.
400:     """A seen video message + its metadata. Drives dedup and future /search.
```

12. assistant (2026-07-06T23:57:00.378Z)

Let me check the backfill call site context (line 291) and its surrounding try/except, and find StageResult in policy.py.

13. user (2026-07-06T23:57:01.886Z)

```
280         matched=matched,
281     )
282     inserted = await db.insert_video(conn, record)
283     log.info(
284         "backfill_video_seen user=%s channel=%s msg=%s matched=%s
reasons=%s",
285         user.id,
286         channel_id,
287         message_id,
288         matched,
```



```

289         reasons,
290     )
291     await _record_policy_decision(
292         conn,
293         inserted=inserted,
294         policy_id=policy_id,
295         policy_passed=policy_passed,
296         policy_results=policy_results,
297     )
298
299     if matched:
300         summary["videos_matched"] += 1
301         dialog_matched += 1
302         link_text = build_link_text(
303             message_link=getattr(message, "link", None),
304             duration_s=meta.duration_s,
305             height=meta.height,
306             size_bytes=meta.size_bytes,
307             channel_title=title, # from the dialog object
308         )
309         await deliver(
310             client,
311             message,
312             conn,
313             inserted.id or 0,
314             delivery_mode,
315             link_text=link_text,
316             target=delivery_target,
317         )
318
319         # Periodic progress (at most once per 30 seconds per dialog).
320         now = asyncio.get_event_loop().time()
321         if progress_callback and now - last_progress > 30:
322             await progress_callback(
323                 f" #{title}: {dialog_total} msgs, {dialog_videos} videos,
{dialog_matched} matched..."
324             )
325             last_progress = now
326
327     except FloodWaitError as e:
328         wait = e.seconds + 1 # small extra buffer
329         log.warning("backfill_flood_wait channel=%s seconds=%s", title, e.seconds)
330         await asyncio.sleep(wait)
331
332     if progress_callback:
333         await progress_callback(
334             f"? #{title}: {dialog_videos} videos, {dialog_matched} matched"
335         )
336
337     return summary
338
339
340 def make_backfill_command_handler( # pragma: no cover - requires live Telethon
connection
341     client: TelegramClient,
342     user: User,
343     conn: aiomysqlite.Connection,
344     engine_holder: dict[str, FilterEngine],
345     delivery_mode: DeliveryMode,
346     delivery_target: str = "me",
347 ) -> Callable:
348     """Return an async handler for ``/backfill [N][hdw]`` commands in Saved Messages.
349
350     The handler runs ``backfill_user`` in a background task so it doesn't
351     block the event loop. Progress messages are sent to ``me``.

```

```

352
353     Matches are delivered to ``delivery_target`` (see ADR 0005).
354
355     ``/stop`` cancels the currently running backfill task (if any).
356
357     ``engine_holder`` is a mutable dict so the backfill picks up any
358     filter config changes made via ``/set`` commands while it runs.
359     """
360
361     _backfill_task: asyncio.Task[None] | None = None
362
363     async def _send_progress(text: str) -> None:
364         try:
365             await client.send_message("me", text)
366         except Exception:
367             log.warning("backfill_progress_send_failed", exc_info=True)
368
369     async def on_command(event: object) -> None:

```

14. user (2026-07-06T23:57:02.926Z)

```

48:     def __init__(self, exceeded_budget: str, **metadata: object) -> None:
139: class StageResult:
145:     metadata: dict[str, object] = field(default_factory=dict)
148:     def pass_(stage_name: str, reason: str, **metadata: object) -> StageResult:
157:     def fail(stage_name: str, reason: str, **metadata: object) -> StageResult:
572:     metadata: dict[str, object] = {
702:     metadata: dict[str, object] = {

```

15. assistant (2026-07-06T23:57:05.792Z)

The finding's claims are structurally confirmed by reading. Now let me confirm the backfill outer `except Exception` at 464 wraps the message loop, and run the repro.

16. user (2026-07-06T23:57:07.806Z)

```

455         await send(
456             f"? Backfill complete: {summary['dialog_count']} channels scanned, "
457             f"{summary['videos_scanned']} videos found, "
458             f"{summary['videos_matched']} matched and delivered"
459         )
460     except asyncio.CancelledError:
461         with contextlib.suppress(Exception):
462             await send("? Backfill cancelled.")
463         raise
464     except Exception:
465         log.exception("backfill_crashed user=%s days=%d", user.id, days)
466         with contextlib.suppress(Exception):
467             await send("? Backfill failed due to an error. Check logs for details.")
468
469
470 __all__ = ["backfill_user", "make_backfill_command_handler"]
471

```

17. user (2026-07-06T23:57:08.378Z)

```

139     class StageResult:
140         """Outcome of one policy stage."""
141
142         passed: bool
143         reason: str
144         stage_name: str
145         metadata: dict[str, object] = field(default_factory=dict)
146

```

```

147     @staticmethod
148     def pass_(stage_name: str, reason: str, **metadata: object) -> StageResult:
149         return StageResult(
150             passed=True,
151             reason=reason,
152             stage_name=stage_name,
153             metadata=metadata,
154         )
155
156     @staticmethod
157     def fail(stage_name: str, reason: str, **metadata: object) -> StageResult:
158         return StageResult(
159             passed=False,
160             reason=reason,
161             stage_name=stage_name,
162             metadata=metadata,
163         )
164
165
166     class PolicyStage(Protocol):
167         """A single stateless gate in the policy pipeline."""
168

```

18. assistant (2026-07-06T23:57:12.346Z)

The `except Exception` at 464 wraps the whole dialog loop (including the try/except FloodWaitError block), so a TypeError from `\_record\_policy\_decision` inside the message loop bubbles past `except FloodWaitError` (327) to 464 and aborts the entire backfill. Confirmed structurally. Now the repro to confirm the serialization crash mechanism.

19. assistant (2026-07-06T23:57:17.781Z)

```

import asyncio, json
from tg_video_filter.policy import StageResult, stage_results_to_trace
from tg_video_filter import db

```

Simulate a stage whose metadata carries a non-JSON-serializable value.

```

results = [StageResult.fail("deep_inspection", "budget exceeded", budget_exceeded=True,
obj=object())]
trace = stage_results_to_trace(results)
print("trace built:", type(trace), [t["metadata"].keys() for t in trace])

```

```

async def main():
    conn = await db.connect(":memory:")
    await db.migrate(conn)
    # need a content_item + policy for FK... but the crash is before any FK check.
    try:
        await db.record_policy_decision(
            conn, content_item_id=1, policy_id=1, passed=False, trace=trace,
        )
        print("NO CRASH")
    except TypeError as e:
        print("record_policy_decision raised TypeError:", e)
    except Exception as e:
        print("other exception:", type(e).__name__, e)

asyncio.run(main())

```

20. user (2026-07-06T23:57:17.783Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

21. user (2026-07-06T23:57:24.640Z)

done

22. user (2026-07-06T23:57:29.116Z)

```
trace built ok, metadata keys: ['budget_exceeded', 'obj']
_policy_trace_json raised TypeError: Object of type object is not JSON serializable
connect ok
```

23. assistant (2026-07-06T23:57:33.838Z)

Repro confirms: `\_policy\_trace\_json` raises an uncaught `TypeError` when stage metadata contains a non-JSON-serializable value. Now let me verify the key mitigating question ? whether any *currently reachable* stage can actually put a non-serializable value into metadata, since that determines whether this is a real shipping bug or latent/defensive.

The `PolicyBudgetExceeded.\_\_init\_\_(self, exceeded\_budget, \*\*metadata)` splats `\*\*exc.metadata` at policy.py:763. Let me check who raises `PolicyBudgetExceeded` and what `.inspect()` does, and whether DeepInspectionStage is actually reachable.

24. user (2026-07-06T23:57:35.339Z)

```
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\db.py:1360:    as ``topics`` /
``security_checks`` / ``deep_inspect`` instead of validating
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\policy.py:45:class
PolicyBudgetExceeded(Exception):
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\policy.py:55:class PolicyBudget:
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\policy.py:72:    async def inspect(self,
fetcher: ContentFetcher, content_item: ContentItem) -> StageResult:
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\policy.py:83:                raise
PolicyBudgetExceeded("timeout_s", timeout_s=self.timeout_s)
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\policy.py:96:                raise
PolicyBudgetExceeded("timeout_s", timeout_s=self.timeout_s)
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\policy.py:100:                raise
PolicyBudgetExceeded("timeout_s", timeout_s=self.timeout_s) from exc
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\policy.py:115:                raise
PolicyBudgetExceeded("timeout_s", timeout_s=configurable_timeout_s)
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\policy.py:122:    async def inspect(self,
content_item: ContentItem, budget: PolicyBudget) -> StageResult:
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\policy.py:341:def
load_deep_inspection_enabled(policy: Policy) -> bool:
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\policy.py:346:    raw_deep_inspect =
decoded.get("deep_inspect", False)
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\policy.py:347:    if
isinstance(raw_deep_inspect, bool):
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\policy.py:348:                return
raw_deep_inspect
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\policy.py:349:    if
isinstance(raw_deep_inspect, dict):
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\policy.py:350:                enabled =
raw_deep_inspect.get("enabled", False)
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\policy.py:353:    raise
ValueError("deep_inspect must be a boolean or an object with boolean enabled")
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\policy.py:728:                enabled =
load_deep_inspection_enabled(ctx.policy)
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\policy.py:738:
invalid_deep_inspection_config=True,
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\policy.py:743:                budget_result =
_deep_inspection_budget_result(ctx)
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\policy.py:779:def
_deep_inspection_budget_result(ctx: PolicyContext) -> StageResult | None:
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\policy.py:868:
"load_deep_inspection_enabled",
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\telethon_client.py:452:    #
DeepInspectionStage is dormant unless ``deep_inspect`` is enabled. Any
```

25. assistant (2026-07-06T23:57:40.283Z)

All present ``PolicyBudgetExceeded`` raises use ``timeout_s=self.timeout_s`` (a float/int ? serializable). And `DeepInspectionStage` requires ``deep_inspect`` enabled AND a `fetcher/budget` wired, which per the code comment at `telethon_client.py:452` is dormant. So the finding's own NOTE is correct: no `*currently reachable*` stage emits non-serializable metadata. This is latent/defensive, not a bug that ships as a live crash today.

Let me verify the `DeepInspectionStage` guard confirms it can't currently produce this path.

26. user (2026-07-06T23:57:41.145Z)

```
720
721     class DeepInspectionStage:
722         """ADR 0012 Tier 4 opt-in boundary for budgeted heavy inspection."""
723
724         stage_name = "deep-inspection"
725
726         async def evaluate(self, ctx: PolicyContext) -> StageResult:
727             try:
728                 enabled = load_deep_inspection_enabled(ctx.policy)
729             except (json.JSONDecodeError, ValueError) as exc:
730                 log.warning(
731                     "deep inspection disabled due to invalid config policy_id=%s",
732                     ctx.policy.id,
733                     exc_info=True,
734                 )
735                 return StageResult.pass_(
736                     self.stage_name,
737                     f"invalid deep inspection config ignored: {exc}",
738                     invalid_deep_inspection_config=True,
739                 )
740             if not enabled:
741                 return StageResult.pass_(self.stage_name, "deep inspection disabled")
742
743             budget_result = _deep_inspection_budget_result(ctx)
744             if budget_result is not None:
745                 return budget_result
746
747             if ctx.fetcher is None:
748                 return StageResult.fail(
749                     self.stage_name,
750                     "deep inspection fetcher missing",
751                     missing_fetcher=True,
752                 )
753
754             assert ctx.budget is not None
755             try:
756                 return await ctx.budget.inspect(ctx.fetcher, ctx.content_item)
757             except PolicyBudgetExceeded as exc:
758                 return StageResult.fail(
759                     self.stage_name,
760                     "deep inspection budget exceeded",
761                     budget_exceeded=True,
762                     exceeded_budget=exc.exceeded_budget,
763                     **exc.metadata,
764                 )
```

27. assistant (2026-07-06T23:57:52.177Z)

Confirmed. In ``_evaluate_policy_stages`` (`telethon_client.py:454`) the ``PolicyContext`` is built with no ``fetcher`/`budget``, so even if ``deep_inspect`` were enabled, ``_deep_inspection_budget_result`` returns a fail (missing budget) or ``ctx.fetcher is None`` returns a fail ? the ``**exc.metadata`` splat path at 763 is unreachable via the wired call sites today. And every present ``PolicyBudgetExceeded`` carries only ``timeout_s`` (numeric). So no live

crash today.

My assessment: The mechanism is real and the repro confirms the uncaught TypeError on the write path. But it is genuinely latent/defensive ? no currently reachable stage can inject non-serializable metadata, exactly as the finding's own NOTE concedes. The write path (``_policy_trace_json``) has no error handling and both call sites lack try/except, so if F5's fetcher wiring lands with untrusted ``**exc.metadata``, a bad value would crash ingestion / abort backfill. This is a legitimate defensive-hardening finding, not a bug shipping wrong behavior in this PR's live paths.

Severity: the finding claims medium. Per the rubric, a **latent** defensive gap that cannot currently be triggered is not a live crash. It matches "note/deviation/growth" more than "real latent bug that crashes ingestion now." However it is a genuine robustness gap on a newly-added write path with a concrete future trigger (F5). I'll rate it low, since it can't occur with current code and the "crashes ingestion" claim doesn't materialize today. Verdict: PLAUSIBLE (mechanism real, but the described wrong behavior ? crashing ingestion ? cannot occur in this PR's reachable code).

28. user (2026-07-06T23:58:03.725Z)

Structured output provided successfully